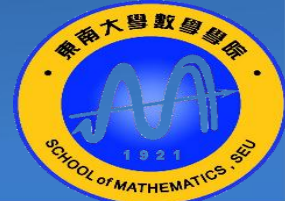


# 常微分方程的数值解



东南大学数学学院 Sch.Math.SEU



在生产和科研中所处理的微分方程往往很复杂且大多得不出解析解。因此，研究常微分方程的数值解法是十分必要的。实际上对初值问题，一般要求得到解在若干个点满足规定精度的近似值，或者得到一个满足精度要求的便于计算的表达式。

常微分方程初值问题  $\begin{cases} x' = f(t, x) \\ x(t_0) = x_0 \end{cases}$  的数值解指从初始点  $t_0$  开始，在指定的一些列离散点处

$$t_0 < t_1 < t_2 < \cdots < t_n$$

求相应的近似值

$$x_1, x_2, \cdots, x_n$$

通常取固定步长  $h$ ,  $t_i = t_0 + ih, i = 0, 1, 2, \cdots n$



# ODE常见数值解法



设  $t_i = t_0 + ih, i = 0, 1, 2, \dots, n$ , 数值求解微分方程: 
$$\begin{cases} x' = f(t, x) \\ x(t_0) = x_0 \end{cases}$$

1. 欧拉法  $x'(t) \approx \frac{x(t+h) - x(t)}{h}$

1.1 向前欧拉法 (欧拉折线法) :  $x_{i+1} = x_i + hf(t_i, x_i)$

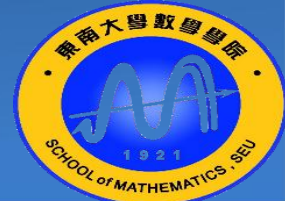
1.2 向后欧拉法:  $x_{i+1} = x_i + hf(t_{i+1}, x_{i+1})$

1.3 梯形法:  $x_{i+1} = x_i + \frac{h}{2}[f(t_i, x_i) + f(t_{i+1}, x_{i+1})]$

1.4 改进的欧拉法: 实际应用时, 与欧拉公式结合使用 (预报 + 校正):

休恩 (Heun) 方法 (预报校正) 
$$\begin{cases} x_{i+1}^{(0)} = x_i + hf(t_i, x_i) \\ x_{i+1}^{(k+1)} = x_i + \frac{h}{2}[f(t_i, y_i) + f(t_{i+1}, x_{i+1}^{(k)})], \quad k = 0, 1, 2, \dots \end{cases}$$

显式算法与隐式算法



## 2. 龙格-库塔法 (Runge-Kutta method)

$$x(t_{i+1}) = x(t_i) + \int_{t_i}^{t_{i+1}} f(s, x(s)) ds = x(t_i) + h\bar{f}$$

$$x_{i+1} = x_i + h \sum_{i=1}^n \omega_i f_i$$

2.1 向前欧拉法 (欧拉折线法)  $x_{i+1} = x_i + hf(t_i, x_i)$

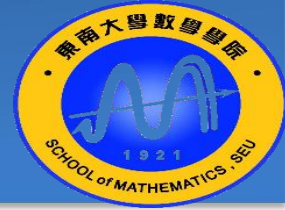
2.2 向后欧拉法  $x_{i+1} = x_i + hf(t_{i+1}, x_{i+1})$

2.3 梯形法  $x_{i+1} = x_i + \frac{h}{2} [f(t_i, x_i) + f(t_{i+1}, x_{i+1})]$

2.4 改进的欧拉法  $x_{i+1} = x_i + \frac{h}{2} [f(t_i, x_i) + f(t_{i+1}, x_i + hf(t_i, x_i))]$



# 2阶、4阶龙格-库塔法



## 2.3 2 阶龙格库塔法 (RK2)/改进的欧拉法/休恩方法

$$\begin{cases} x_{i+1} = x_i + \frac{h}{2}(k_1 + k_2) \\ k_1 = f(t_i, x_i) \\ k_2 = f(t_{i+1}, x_i + hk_1) \end{cases}$$

## 2.4 4 阶龙格库塔法 (RK4)

$$\begin{cases} x_{i+1} = x_i + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \\ k_1 = f(t_i, x_i) \\ k_2 = f(t_i + \frac{1}{2}h, x_i + \frac{1}{2}hk_1) \\ k_3 = f(t_i + \frac{1}{2}h, x_i + \frac{1}{2}hk_2) \\ k_4 = f(t_{i+1}, x_i + hk_3) \end{cases}$$



# 数值算法的精度



假设  $x_i = x(t_i), i = 0, 1, 2, \dots, n-1$ , 则称  $R_n(h) = x(t_n) - x_n$  为数值解法在第  $n$  步的局部截断误差

称  $E_n(h) = \sum_{i=1}^n R_i(h)$  为数值解法在  $t_n$  点上的整体截断误差

解法具有  $p$  阶精度或是  $p$  阶算法

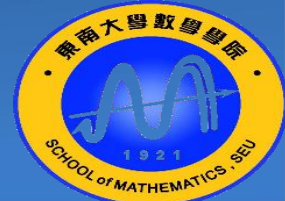
- $R_n(h) = O(h^{p+1})$
- $E_n(h) = O(h^p)$

$p$  越大, 则数值算法的精度越高。

1. 向前/向后欧拉法是一阶算法, 改进的欧拉法与梯形法是二阶算法。
2. 龙格-库塔法常用的有二阶、四阶等公式。



# 用Matlab软件求常微分方程的数值解



MATLAB 做 ODE 模型仿真

```
[t,x]=solver (@odesys,ts,x0,options)
```

输入参数:

- solver, ODE 仿真命令, 常用 ode45,ode23,ode15s,ode23s 等, 一般模型首先 ode45, 采用变步长的 5 级 4 阶龙格-库塔-芬尔格算法. *s* 结尾的命令用于刚性方程
- @odesys, 其中 odesys 为定义 ODE 模型的函数名
- $ts = [t_0 : dt : t_f]$ , 其中  $t_0, dt, t_f$  分别为自变量初值, 步长及终值, 步长缺省时,  $ts = [t_0 t_f]$ , 由 Matlab 自定义分割点
- $x_0$  表示状态变量  $x$  的初值
- options, 可选参数, 通常用于设定误差 (默认时相对误差  $rt = 10^{-3}$ , 绝对误差  $at = 10^{-6}$ ), 自定义误差限时使用命令 `options=odeset('reltol',rt,'abstol',at)`, 其中  $rt, at$  分别为设定的相对误差和绝对误差.

返回值:

- $t$ , 表示在分割点处的自变量构成的列向量
- $x$ , 表示在分割点处的自变量构成的矩阵,  $x$  的第  $i$  列表示  $x_i$  的计算值





# 方程组与高阶方程



## 注意事项

1. Matlab 解  $n$  维常微分方程组时, 变量  $x$  为  $n$  维向量, 定义模型的函数中应以分量形式返回  $x$  的一阶导数信息.
2. 使用 Matlab 软件求 ODE 数值解时, 高阶微分方程需要等价地变换成一阶微分方程组.

$n$  阶常微分方程模型转化为  $n$  维一阶常微分方程组模型:

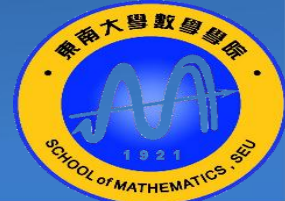
Step 1. 从  $n$  阶常微分方程模型  $F(t, x, x', \dots, x^{(n)}) = 0$  中解出最高阶导数  $x^{(n)} = f(t, x, x', \dots, x^{(n-1)})$

Step 2. 令  $y = (x, x', \dots, x^{(n-1)})^T$ , 则得到等价的一阶常微分方程组模型

$$y' = g(t, y) = (y_2, y_3, \dots, f(t, y))^T$$

$$\begin{cases} y'_1 = y_2 \\ y'_2 = y_3 \\ \vdots \\ y'_{n-1} = y_n \\ y'_n = f(t, y) \end{cases}$$





例. 在区间  $[0, 20]$  上求模型的数值解

$$\begin{cases} \frac{d^2x}{dt^2} + (x^2 - 1) \frac{dx}{dt} + x = 0 \\ x(0) = 2; x'(0) = 0 \end{cases}$$

解: 令  $y_1 = x, y_2 = x'$ , 将二阶微分方程变为等价的一阶微分方程组:

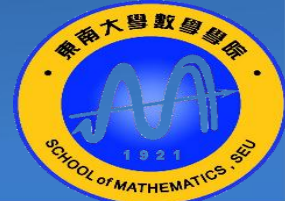
$$\begin{cases} y_1' = y_2 \\ y_2' = (1 - y_1^2)y_2 - y_1 \\ y_1(0) = 2, y_2(0) = 0 \end{cases}$$

1. MATLAB 定义 ODE 模型函数 **vdp**

```
function dy=vdp(t,y)
    dy=zeros(2,1);
    dy(1)=y(2);
    dy(2)=(1-y(1)^2)*y(2)-y(1);
end
```

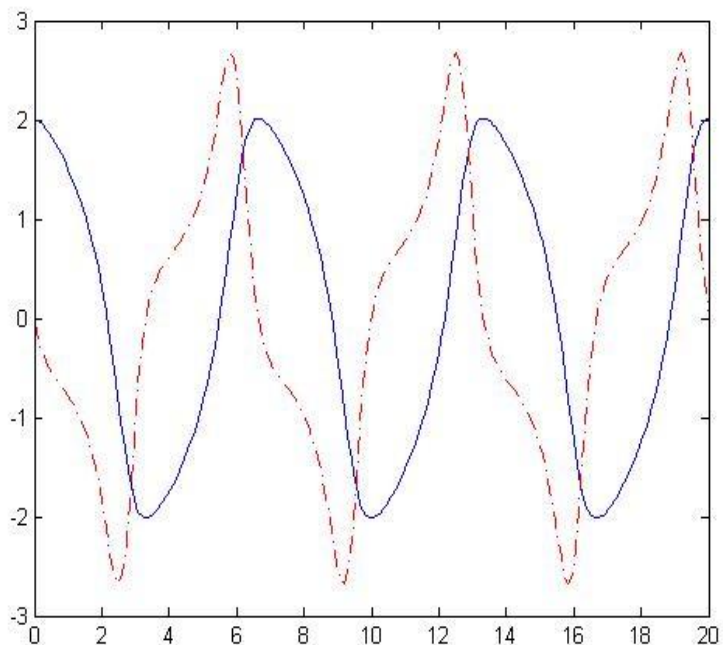
2. MATLAB 仿真 ODE 模型, 取  $t_0 = 0, t_f = 20$

```
[t,y]=ode45('vdp',[0 20],[2 0]);
figure(1)
plot(t,y(:,1),'b-')
hold on
plot(t,y(:,2),'r-.')
hold off
figure(2)
plot(y(:,1),y(:,2),'b-')
```



## 1. MATLAB 定义 ODE 模型函数 **vdp**

```
function dy=vdp(t,y)
    dy=zeros(2,1);
    dy(1)=y(2);
    dy(2)=(1-y(1)^2)*y(2)-y(1);
end
```



## 2. MATLAB 仿真 ODE 模型, 取 $t_0 = 0$ $t_f = 20$

```
[t,y]=ode45('vdp',[0 20],[2 0]);
figure(1)
plot(t,y(:,1),'b-')
hold on
plot(t,y(:,2),'r-.')
hold off
figure(2)
plot(y(:,1),y(:,2),'b-')
```

